

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 12 – SEARCHING



NAMA: Aghneo Amar Ahsan

NIM: 109082500118

KELAS S1IF-13-05

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

1. Konsep Dasar Pencarian

Pencarian data pada pemrograman dilakukan untuk menemukan informasi yang lebih spesifik, seperti mencari nama mahasiswa atau NIM tertentu. Berbeda dengan kasus pencarian nilai ekstrim (seperti mencari nilai maksimum atau minimum) yang datanya selalu bisa ditemukan, proses pencarian data spesifik ini memiliki kemungkinan bahwa data yang dicari tidak ada di dalam kumpulan data tersebut. Selain itu, ada juga beberapa algoritma pencarian yang bekerja dengan cara memanfaatkan keterurutan sebuah data agar proses pencariannya menjadi lebih efisien.

2. Sequential Search (Pencarian Sekuensial)

Algoritma Sequential Search bekerja dengan mengecek elemen data secara satu per satu dan berurutan, mulai dari data pertama hingga data terakhir. Ciri khas dari algoritma ini adalah proses pencariannya akan langsung berhenti ketika data yang dicari sudah berhasil ditemukan, walaupun di belakangnya masih banyak sisa data yang belum dicek. Secara teknis, algoritma ini memakai sebuah status (biasanya berupa variabel dengan tipe boolean, misalnya found) untuk menandai apakah datanya sudah ketemu atau belum. Perulangan di dalam program wajib dihentikan apabila status pencarian sudah bernilai true (data ditemukan) atau elemen data yang posisinya paling akhir telah selesai dicek. Karena dalam pemrograman lokasi indeks dari suatu data seringkali jauh lebih penting daripada isi data itu sendiri, algoritma ini biasa dimodifikasi agar bisa menghasilkan nilai indeks tempat data tersebut berada. Jika datanya tidak ditemukan, program biasanya akan disetel untuk menghasilkan angka -1 atau bilangan lain yang berada di luar indeks array yang valid.

3. Binary Search (Pencarian Biner)

Syarat mutlak untuk bisa memakai algoritma Binary Search adalah datanya harus sudah dalam keadaan terurut, baik itu dari kecil ke besar (ascending) maupun dari besar ke kecil (descending). Cara kerjanya dimulai dengan mengambil nilai dari data yang posisinya berada paling tengah dalam suatu rentang pencarian. Jika data tengah yang terambil itu ternyata nilainya terlalu kecil, maka rentang pencarian akan otomatis digeser ke sebelah kanan. Hal ini sangat logis karena jika data yang di tengah saja kurang besar, maka semua data di sebelah kirinya juga pasti akan bernilai terlalu kecil dari data yang sedang dicari. Begitu juga sebaliknya, jika data yang terambil nilainya terlalu besar, maka rentang pencariannya akan digeser ke sisi kiri. Algoritma ini tidak akan bisa berjalan kalau cara pengurutannya terbalik, misalnya memakai rumusan ascending untuk mencari sekumpulan data yang disusun secara descending. Proses Binary Search ini akan berhenti

beroperasi kalau batas indeks kirinya sudah melampaui indeks kanan (yang menandakan datanya memang tidak ada), atau ketika datanya sudah berhasil ditemukan.

4. Pencarian pada Array Bertipe Data Struct

Konsep pencarian pada tipe data struct sebenarnya tidak jauh berbeda dengan pencarian pada tipe data dasar, hanya saja kita harus menambahkan field atau kategori khusus dari pencarian yang mau dilakukan. Khusus untuk algoritma Binary Search, urutan array di dalamnya harus sama persis dengan field kategori pencariannya. Sebagai contoh, kalau array sudah disusun berurutan berdasarkan NIM mahasiswa, maka algoritma Binary Search cuma bisa dipakai kalau kita mencari berdasarkan NIM juga. Algoritma tersebut tidak akan jalan kalau array-nya terurut berdasarkan NIM, tapi kita malah melakukan pencarian menggunakan field Nama. Untuk mengatasi masalah ini, array tersebut harus diurutkan ulang berdasarkan field yang mau dicari terlebih dahulu, atau solusi paling gampangnya adalah langsung saja gunakan algoritma Sequential Search.

B. Guided

1. Sequential Search

```
package main

import "fmt"

func SequnsialSearch(arr []string, dataCari string) int {
    var idx_found int = -1
    for i := 0; i < len(arr); i++ {
        if arr[i] == dataCari {
            idx_found = i
            break
        }
    }
    return idx_found
}

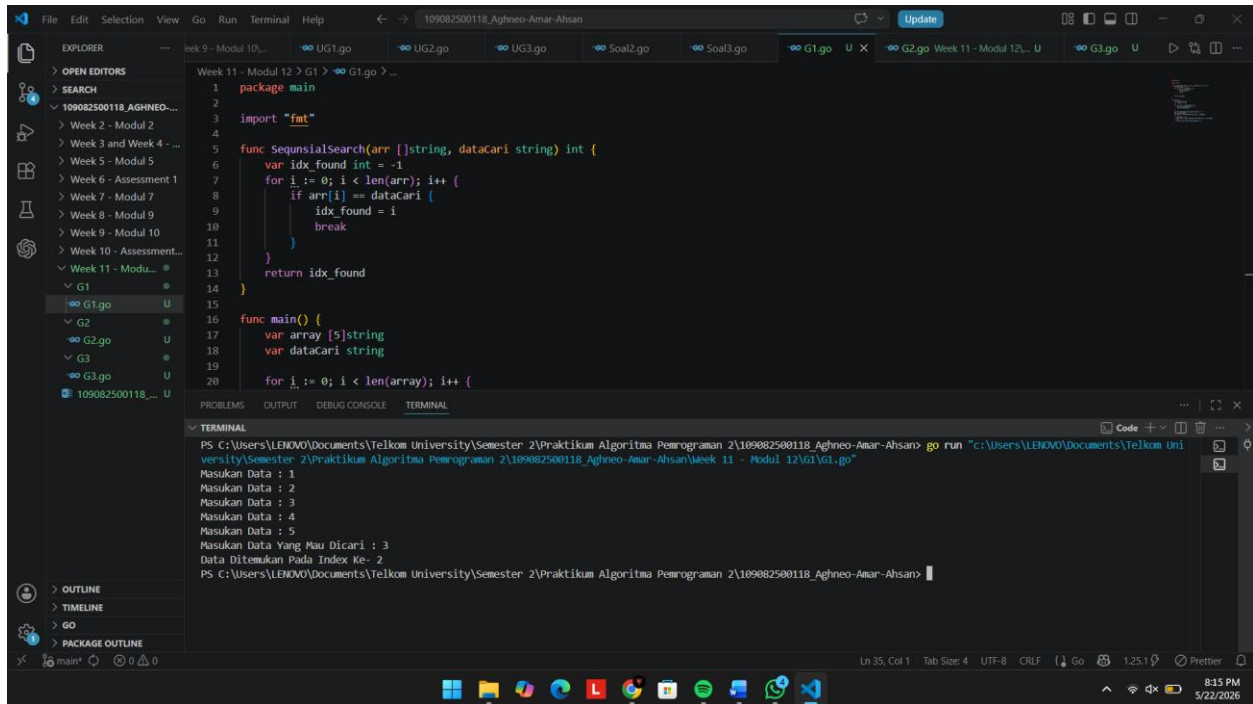
func main() {
    var array [5]string
    var dataCari string

    for i := 0; i < len(array); i++ {
        fmt.Print("Masukan Data : ")
        fmt.Scan(&array[i])
    }

    fmt.Print("Masukan Data Yang Mau Dicari : ")
    fmt.Scan(&dataCari)
    idx_found := SequnsialSearch(array[:], dataCari)

    if idx_found > -1 {
        fmt.Println("Data Ditemukan Pada Index Ke-",
idx_found)
    } else {
        fmt.Println("Data Tidak Ditemukan!!!")
    }
}
```

Output :



The screenshot shows a VS Code editor with a Go file named `G1.go`. The code implements a sequential search function. The terminal output shows the program being run, with prompts for data input and the search result.

```
1 package main
2
3 import "fmt"
4
5 func SequentialSearch(arr []string, dataCari string) int {
6     var idx_found int = -1
7     for i := 0; i < len(arr); i++ {
8         if arr[i] == dataCari {
9             idx_found = i
10            break
11        }
12    }
13    return idx_found
14 }
15
16 func main() {
17     var array [5]string
18     var dataCari string
19
20     for i := 0; i < len(array); i++ {
```

Terminal Output:

```
PS C:\Users\LENOVO\Documents\Telkom University\Semester 2\Praktikum Algoritma Pemrograman 2\109082500118_Aghneo-Amar-Ahsan> go run "c:\Users\LENOVO\Documents\Telkom University\Semester 2\Praktikum Algoritma Pemrograman 2\109082500118_Aghneo-Amar-Ahsan\Week 11 - Modul 12\G1\G1.go"
Masukan Data : 1
Masukan Data : 2
Masukan Data : 3
Masukan Data : 4
Masukan Data : 5
Masukan Data Yang Mau Dicari : 3
Data Ditemukan Pada Index Ke- 2
PS C:\Users\LENOVO\Documents\Telkom University\Semester 2\Praktikum Algoritma Pemrograman 2\109082500118_Aghneo-Amar-Ahsan>
```

2. Nilai Ekstrim Pada Struct

```
package main

import "fmt"

const max = 100

type arr [max]int

func BinarySearch(a arr, n int, x int) bool {
    var kiri int = 0
    var kanan int = n - 1
    var found bool = false
    var tengah int

    for kiri <= kanan && !found {
        tengah = (kiri + kanan) / 2
        if a[tengah] < x {
            kiri = tengah + 1
        } else if a[tengah] > x {
            kanan = tengah - 1
        } else {
            found = true
        }
    }
}
```

```

        return found
    }

func main() {
    var data arr
    var n, x int

    fmt.Println("Input Jumlah Data : ")
    fmt.Scan(&n)

    fmt.Println("Input Data Ascending : ")
    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }

    fmt.Println("Input Data Yang Dicari : ")
    fmt.Scan(&x)

    if BinarySearch(data, n, x) {
        fmt.Println("Data Ditemukan!!!!")
    } else {
        fmt.Println("Data Tidak Ditemukan!!!!")
    }
}

```

Output :

The screenshot shows a Visual Studio Code editor with a Go file named `G2.go` open. The code implements a binary search algorithm. The terminal window at the bottom shows the execution of the program with the following output:

```

PS C:\Users\LENOVO\Documents\Telkom University\Semester 2\Praktikum Algoritma Pemrograman 2\109082500118_Aghneo-Amar-Ahsan> go run "c:\Users\LENOVO\Documents\Telkom University\Semester 2\Praktikum Algoritma Pemrograman 2\109082500118_Aghneo-Amar-Ahsan\Week 11 - Modul 12\G2.go"
Input Jumlah Data : 3
Input Data Ascending : 1 2 3
Input Data Yang Dicari : 2
Data Ditemukan!!!!
PS C:\Users\LENOVO\Documents\Telkom University\Semester 2\Praktikum Algoritma Pemrograman 2\109082500118_Aghneo-Amar-Ahsan>

```

3. Nilai Ekstrim Pada Struct

```
package main

import "fmt"

type mahasiswa struct {
    nama string
    nim   int
    IPK   float64
}

type arrMahasiswa [5]mahasiswa

func binSearchStruct(arrData arrMahasiswa, nimDicari int)
int {
    var found int = -1
    var med int
    var kr int = 0
    var kn int = len(arrData) - 1
    for kr <= kn && found == -1 {
        med = (kr + kn) / 2
        if nimDicari < arrData[med].nim { //kurang dari
median, pencarian ke kiri
            kn = med - 1
        } else if nimDicari > arrData[med].nim { //lebih
dari median, pencarian ke kanan
            kr = med + 1
        } else {
            found = med
        }
    }
    return found
}

func seqSearchStruct(arrData arrMahasiswa, nimDicari int)
int {
    var found int = -1
    for i := 0; i < len(arrData); i++ {
        if arrData[i].nim == nimDicari {
            found = i
            break
        }
    }
    return found
}

func main() {
    var mahasiswa06 arrMahasiswa
    var nimDicari int
```

```

    fmt.Println("--- MASUKKAN DATA NIM MAHASISWA SECARA
TERURUT ---")
    for i := 0; i < len(mahasiswa06); i++ {
        fmt.Printf("=== Masukkan data mahasiswa 06 pada
indeks ke-%d ===\n", i)
        fmt.Print("Masukkan nama : ")
        fmt.Scan(&mahasiswa06[i].nama)
        fmt.Print("Masukkan NIM : ")
        fmt.Scan(&mahasiswa06[i].nim)
        fmt.Print("Masukkan IPK : ")
        fmt.Scan(&mahasiswa06[i].IPK)
        fmt.Println("-----")
    }
    fmt.Println()

    fmt.Print("Masukkan NIM mahasiswa yang ingin dicari : ")
    fmt.Scan(&nimDicari)
    fmt.Println()

    var idxHasilCariSeqSearch int
    idxHasilCariSeqSearch = seqSearchStruct(mahasiswa06,
nimDicari)

    fmt.Println("== HASIL SEQUENTIAL SEARCH ==")
    if idxHasilCariSeqSearch > -1 {
        fmt.Printf("Data NIM mahasiswa %d ditemukan pada
array di indeks ke-%d!\n", nimDicari, idxHasilCariSeqSearch)
        fmt.Println("Berikut Detail Datanya : ")
        fmt.Printf("Nama : %s\n",
mahasiswa06[idxHasilCariSeqSearch].nama)
        fmt.Printf("NIM : %d\n",
mahasiswa06[idxHasilCariSeqSearch].nim)
        fmt.Printf("IPK : %.2f\n",
mahasiswa06[idxHasilCariSeqSearch].IPK)
    } else if idxHasilCariSeqSearch == -1 {
        fmt.Printf("Data NIM mahasiswa %d tidak ditemukan
pada array!", nimDicari)
    }
    fmt.Println()

    var idxHasilCariBinSearch int
    idxHasilCariBinSearch = binSearchStruct(mahasiswa06,
nimDicari)

    fmt.Println("== HASIL BINARY SEARCH ==")
    if idxHasilCariBinSearch > -1 {
        fmt.Printf("Data NIM mahasiswa %d ditemukan pada
array di indeks ke-%d!\n", nimDicari, idxHasilCariBinSearch)
        fmt.Println("Berikut Detail Datanya : ")

```

```

        fmt.Printf("Nama : %s\n",
mahasiswa06[idxHasilCariBinSearch].nama)
        fmt.Printf("NIM : %d\n",
mahasiswa06[idxHasilCariBinSearch].nim)
        fmt.Printf("IPK : %.2f\n",
mahasiswa06[idxHasilCariBinSearch].IPK)
    } else if idxHasilCariBinSearch == -1 {
        fmt.Printf("Data NIM mahasiswa %d tidak ditemukan
pada array!", nimDicari)
    }
}

```

Output :

```

Masukkan IPK : 3.1
===== Masukkan data mahasiswa 06 pada indeks ke-1 =====
Masukkan nama : bb
Masukkan NIM : 23
Masukkan IPK : 3.2
=====
===== Masukkan data mahasiswa 06 pada indeks ke-2 =====
Masukkan nama : cc
Masukkan NIM : 34
Masukkan IPK : 3.3
=====
===== Masukkan data mahasiswa 06 pada indeks ke-3 =====
Masukkan nama : dd
Masukkan NIM : 45
Masukkan IPK : 3.4
=====
===== Masukkan data mahasiswa 06 pada indeks ke-4 =====
Masukkan nama : ee
Masukkan NIM : 56
Masukkan IPK : 3.5
=====

Masukkan NIM mahasiswa yang ingin dicari : 23

== HASIL SEQUENTIAL SEARCH ==
Data NIM mahasiswa 23 ditemukan pada array di indeks ke-11
Berikut Detail Datanya :
Nama : bb
NIM : 23
IPK : 3.20

== HASIL BINARY SEARCH ==
Data NIM mahasiswa 23 ditemukan pada array di indeks ke-11
Berikut Detail Datanya :
Nama : bb
NIM : 23
IPK : 3.20
PS C:\Users\LEHONO\Documents\Telkom University\Semester 2\Praktikum Algoritma Pemrograman 2\109082500118_Aghneo-Amar-Ahsan>

```

C. Unguided

1. Validasi Suara Pilkart

- 1) Pada pemilihan ketua RT yang baru saja berlangsung, terdapat 20 calon ketua yang bertanding memperebutkan suara warga. Perhitungan suara dapat segera dilakukan karena warga cukup mengisi formulir dengan nomor dari calon ketua RT yang dipilihnya. Seperti biasa, selalu ada pengisian yang tidak tepat atau dengan nomor pilihan di luar yang tersedia, sehingga data juga harus divalidasi. Tugas Anda untuk membuat program mencari siapa yang memenangkan pemilihan ketua RT.

Buatlah program **pilkart** yang akan membaca, memvalidasi, dan menghitung suara yang diberikan dalam pemilihan ketua RT tersebut.

Masukan hanya satu baris data saja, berisi bilangan bulat valid yang kadang tersisipi dengan data tidak valid. Data valid adalah integer dengan nilai di antara 1 s.d. 20 (inklusif). Data berakhir jika ditemukan sebuah bilangan dengan nilai 0.

Keluaran dimulai dengan baris berisi jumlah data suara yang terbaca, diikuti baris yang berisi berapa banyak suara yang valid. Kemudian sejumlah baris yang mencetak data para calon apa saja yang mendapatkan suara.

No	Masukan	Keluaran
1	7 19 3 2 78 3 1 -3 18 19 0	Suara masuk: 10 Suara sah: 8 1: 1 2: 1 3: 2 7: 1 18: 1 19: 2

Jawaban :

```
package main

import "fmt"

func main() {
    var vote int
    var totalSuara, suaraSah int
    var counts [21]int

    for {
        fmt.Scan(&vote)
        if vote == 0 {
            break
        }
    }
}
```

```

    }

    totalSuara++

    if vote >= 1 && vote <= 20 {
        suaraSah++
        counts[vote]++
    }
}

fmt.Printf("Suara masuk: %d\n", totalSuara)
fmt.Printf("Suara sah: %d\n", suaraSah)
for i := 1; i <= 20; i++ {
    if counts[i] > 0 {
        fmt.Printf("%d: %d\n", i, counts[i])
    }
}
}

```

Output :

The screenshot shows a VS Code editor with a Go file named `UG1.go` open. The code in the editor is as follows:

```

1 package main
2
3 import "fmt"
4
5 func main() {
6     var vote int
7     var totalSuara, suaraSah int
8     var counts [21]int
9
10    for {
11        fmt.Scan(&vote)
12        if vote == 0 {
13            break
14        }
15
16        totalSuara++
17
18        if vote >= 1 && vote <= 20 {
19            suaraSah++
20        }
21    }
22
23    fmt.Printf("Suara masuk: %d\n", totalSuara)
24    fmt.Printf("Suara sah: %d\n", suaraSah)
25    for i := 1; i <= 20; i++ {
26        if counts[i] > 0 {
27            fmt.Printf("%d: %d\n", i, counts[i])
28        }
29    }
30 }

```

The terminal output shows the execution of the program:

```

PS C:\Users\LEHNO\Documents\Telkom University\Semester 2\Praktikum Algoritma Pemrograman 2\109082500118_Aghneo-Amar-Ahsan> go run "c:\Users\LEHNO\Documents\Telkom University\Semester 2\Praktikum Algoritma Pemrograman 2\109082500118_Aghneo-Amar-Ahsan\Week 11 - Modul 12\UG1.go"
7 19 3 2 78 3 1 -3 18 19 0
Suara masuk: 10
Suara sah: 8
1: 1
2: 1
3: 2
7: 1
18: 1
19: 2

```

Penjelasan :

Program ini dirancang untuk menghitung hasil pemungutan suara dengan membaca deretan angka secara berulang hingga menemukan angka 0 sebagai penanda berhentinya proses input. Setiap angka yang dimasukkan akan dihitung sebagai "Suara masuk", namun hanya angka dalam rentang 1 hingga 20 yang akan divalidasi sebagai "Suara sah" dan direkap ke dalam tempat

penyimpanan data perolehan suara masing-masing kandidat. Berdasarkan tangkapan layar, dari input deretan angka 7 19 3 2 78 3 1 -3 18 19 0, terdapat total 10 angka yang masuk sebelum angka 0, sehingga menghasilkan output "Suara masuk: 10". Dari sepuluh angka tersebut, angka 78 dan -3 diabaikan karena berada di luar batas rentang valid, sehingga hanya menyisakan 8 suara sah ("Suara sah: 8"). Kedelapan suara sah tersebut kemudian direkap secara otomatis dan ditampilkan berurutan dari nomor kandidat terkecil yang mendapat suara, yang menghasilkan rincian perolehan untuk kandidat nomor urut 1 (1 suara), 2 (1 suara), 3 (2 suara), 7 (1 suara), 18 (1 suara), dan 19 (2 suara).

2. Pemenang Pilkart

2) Berdasarkan program sebelumnya, buat program **pilkart** yang mencari siapa pemenang pemilihan ketua RT. Sekaligus juga ditentukan bahwa wakil ketua RT adalah calon yang mendapatkan suara terbanyak kedua. Jika beberapa calon mendapatkan suara terbanyak yang

sama, ketua terpilih adalah dengan nomor peserta yang paling kecil dan wakilnya dengan nomor peserta terkecil berikutnya.

Masukan hanya satu baris data saja, berisi bilangan bulat valid yang kadang tersisipi dengan data tidak valid. Data valid adalah bilangan bulat dengan nilai di antara 1 s.d. 20 (inklusif). Data berakhir jika ditemukan sebuah bilangan dengan nilai 0.

Keluaran dimulai dengan baris berisi jumlah data suara yang terbaca, diikuti baris yang berisi berapa banyak suara yang valid. Kemudian tercetak calon nomor berapa saja yang menjadi pasangan ketua RT dan wakil ketua RT yang baru.

No	Masukan	Keluaran
1	7 19 3 2 78 3 1 -3 18 19 0	Suara masuk: 10 Suara sah: 8 Ketua RT: 3 Wakil ketua: 19

Jawaban :

```
package main

import "fmt"

func main() {
    var vote int
    var totalSuara, suaraSah int
    var counts [21]int
```

```

    for {
        fmt.Scan(&vote)
        if vote == 0 {
            break
        }

        totalSuara++

        if vote >= 1 && vote <= 20 {
            suaraSah++
            counts[vote]++
        }
    }

    var ketua, wakil int

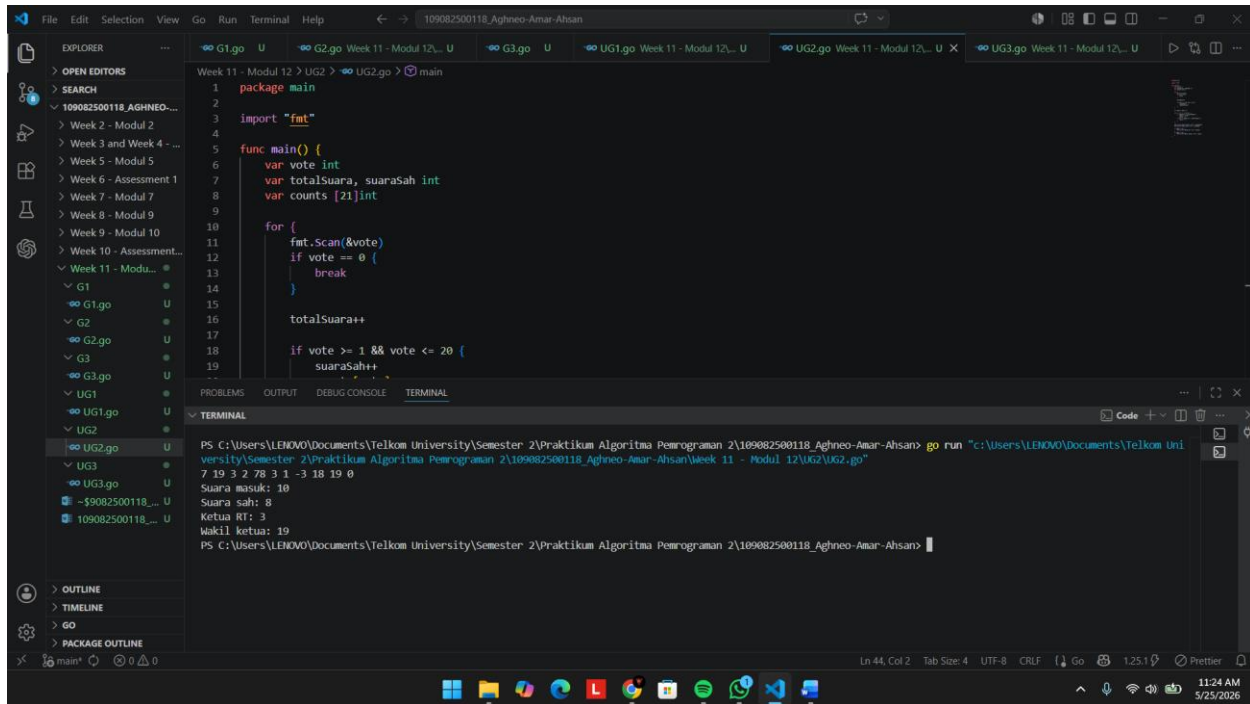
    for i := 1; i <= 20; i++ {
        if counts[i] > counts[ketua] {
            wakil = ketua
            ketua = i
        } else if counts[i] > counts[wakil] {
            wakil = i
        }
    }

    fmt.Printf("Suara masuk: %d\n", totalSuara)
    fmt.Printf("Suara sah: %d\n", suaraSah)

    if ketua != 0 {
        fmt.Printf("Ketua RT: %d\n", ketua)
    }
    if wakil != 0 {
        fmt.Printf("Wakil ketua: %d\n", wakil)
    }
}

```

Output :



```
1 package main
2
3 import "fmt"
4
5 func main() {
6     var vote int
7     var totalSuara, suaraSah int
8     var counts [21]int
9
10    for {
11        fmt.Scan(&vote)
12        if vote == 0 {
13            break
14        }
15
16        totalSuara++
17
18        if vote >= 1 && vote <= 20 {
19            suaraSah++
20        }
21    }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

PS C:\Users\LEHMO\Documents\Telkom University\Semester 2\Praktikum Algoritma Pemrograman 2\109082500118_Aghneo-Amar-Ahsan> go run "c:\Users\LEHMO\Documents\Telkom University\Semester 2\Praktikum Algoritma Pemrograman 2\109082500118_Aghneo-Amar-Ahsan\Week 11 - Modul 12\UG2.go"

7 19 3 2 78 3 1 -3 18 19 0
Suara masuk: 10
Suara sah: 8
Ketua RT: 3
Wakil ketua: 19
PS C:\Users\LEHMO\Documents\Telkom University\Semester 2\Praktikum Algoritma Pemrograman 2\109082500118_Aghneo-Amar-Ahsan>

Penjelasan :

Program ini dibuat untuk mencari pemenang dan wakil dalam pemilihan ketua RT dengan membaca data perolehan suara yang masuk secara berulang. Sesuai dengan aturan pemilu tersebut, data yang dinilai valid hanyalah angka dari 1 sampai 20, dan proses input akan langsung berhenti ketika program membaca angka 0. Setiap angka yang diketik akan selalu dihitung sebagai total suara masuk, tetapi hanya angka yang valid saja yang akan dihitung sebagai suara sah dan ditambahkan ke dalam tempat penyimpanan data rekapitulasi (array). Setelah proses penginputan selesai, program akan membandingkan isi dari tempat penyimpanan tersebut untuk mencari kandidat dengan suara terbanyak pertama sebagai ketua dan suara terbanyak kedua sebagai wakilnya. Berdasarkan tangkapan layar, ketika program diberikan masukan berupa deretan angka 7 19 3 2 78 3 1 -3 18 19 0, ada 10 angka yang terbaca sebelum penanda angka 0, sehingga mencetak keluaran "Suara masuk: 10". Dari sepuluh angka tersebut, nilai 78 dan -3 ditolak karena berada di luar batasan angka kandidat, sehingga hanya tersisa 8 angka valid yang kemudian mencetak

keluaran "Suara sah: 8". Saat perolehan suaranya direkap, kandidat nomor 3 dan nomor 19 ternyata sama-sama mendapatkan perolehan tertinggi yaitu 2 suara, namun karena program mengecek dari nomor peserta paling kecil terlebih dahulu, maka kandidat 3 otomatis mengisi posisi utama sebagai "Ketua RT: 3" dan kandidat 19 bergeser mengisi posisi kedua sebagai "Wakil ketua: 19".

3. Binary Search

3) Diberikan n data integer positif dalam keadaan terurut membesar dan sebuah integer lain k , apakah bilangan k tersebut ada dalam daftar bilangan yang diberikan? Jika ya, berikan indeksnya, jika tidak sebutkan "TIDAK ADA".

Masukan terdiri dari dua baris. Baris pertama berisi dua buah integer positif, yaitu n dan k . n menyatakan banyaknya data, dimana $1 < n \leq 1000000$. k adalah bilangan yang ingin dicari. Baris kedua berisi n buah data integer positif yang sudah terurut membesar.

Keluaran terdiri dari satu baris saja, yaitu sebuah bilangan yang menyatakan posisi data yang dicari (k) dalam kumpulan data yang diberikan. Posisi data dihitung dimulai dari angka 0. Atau memberikan keluaran "TIDAK ADA" jika data k tersebut tidak ditemukan dalam kumpulan.

Program yang dibangun harus menggunakan subprogram dengan mengikuti kerangka yang sudah diberikan berikut ini.

```
package main
import "fmt"

const NMAX = 1000000
var data [NMAX]int

func main(){
    /* buatlah kode utama yang membaca baris pertama (n dan k). kemudian data diisi
    oleh prosedur isiArray(n), dan pencarian oleh fungsi posisi(n,k), dan
    setelah itu output dicetak. */
}
```

```
func isiArray(n int){
    /* I.S. terdefinisi integer n, dan sejumlah n data sudah siap pada piranti
    masukan.
    F.S. Array data berisi n (<=NMAX) bilangan */
}

func posisi(n, k int) int {
    /* mengembalikan posisi k dalam array data dengan n elemen. Posisi dimulai
    dari posisi 0. Jika tidak ada kembalikan -1 */
}
```

Contoh masukan dan keluaran:

No	Masukan	Keluaran	Penjelasan
1	12 534 1 3 8 16 32 123 323 323 534 543 823 999	8	Data 534 berada pada posisi ke-8 dihitung dari awal data.
2	12 535 1 3 8 16 32 123 323 323 534 543 823 999	TIDAK ADA	

Jawaban :

```
package main
```

```

import "fmt"

const NMAX = 1000000

var data [NMAX]int

func main() {
    var n, k int
    fmt.Scan(&n, &k)

    isiArray(n)

    hasil := posisi(n, k)
    if hasil == -1 {
        fmt.Println("TIDAK ADA")
    } else {
        fmt.Println(hasil)
    }
}

func isiArray(n int) {
    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }
}

func posisi(n, k int) int {

    kr := 0
    kn := n - 1

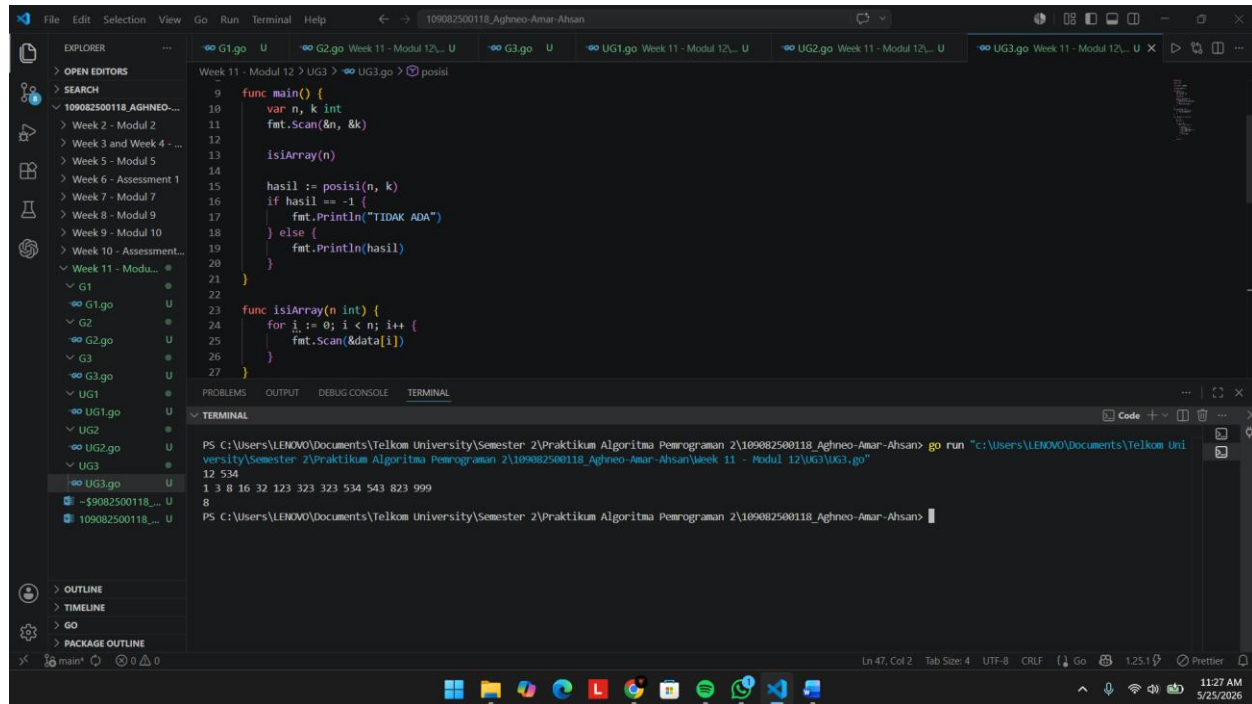
    for kr <= kn {
        med := (kr + kn) / 2

        if data[med] < k {
            kr = med + 1
        } else if data[med] > k {
            kn = med - 1
        } else {
            return med
        }
    }

    return -1
}

```

Output :



The screenshot shows a Go IDE with a file explorer on the left, a code editor in the center, and a terminal at the bottom. The code editor displays a Go program for binary search. The terminal shows the execution of the program, which reads the number of elements (12) and the target (534), then prints the index of the target (8) if found, or "TIDAK ADA" if not found.

```
func main() {
    var n, k int
    fmt.Scan(&n, &k)

    isiArray(n)

    hasil := posisi(n, k)
    if hasil == -1 {
        fmt.Println("TIDAK ADA")
    } else {
        fmt.Println(hasil)
    }
}

func isiArray(n int) {
    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }
}
```

Terminal Output:

```
PS C:\Users\LENOVO\Documents\Telkom University\Semester 2\Praktikum Algoritma Pemrograman 2\109082500118_Aghneo-Amar-Ahsan> go run "c:\Users\LENOVO\Documents\Telkom University\Semester 2\Praktikum Algoritma Pemrograman 2\109082500118_Aghneo-Amar-Ahsan\Week 11 - Modul 12\UG3\UG3.go"
12 534
1 3 8 16 32 123 323 323 534 543 823 999
8
PS C:\Users\LENOVO\Documents\Telkom University\Semester 2\Praktikum Algoritma Pemrograman 2\109082500118_Aghneo-Amar-Ahsan>
```

Penjelasan :

Program ini dirancang menggunakan metode pencarian biner (Binary Search) untuk menemukan posisi letak suatu angka spesifik di dalam sekumpulan data yang sudah disusun secara berurutan. Prosesnya dimulai dengan membaca dua buah nilai awal terlebih dahulu, yaitu jumlah total rentetan data dan angka target yang ingin dicari posisinya. Setelah itu, fungsi pendukung bernama `isiArray` akan bertugas membaca barisan angka masukan dan menyimpannya ke dalam daftar penyimpanan program. Tahap pencarian utama kemudian diambil alih oleh fungsi `posisi` yang bekerja membelah rentang pencarian menjadi dua bagian secara terus-menerus—mengecek titik tengahnya untuk menyeleksi apakah angka target berada di separuh kiri atau separuh kanan—hingga angkanya berhasil ditemukan atau seluruh rentangnya benar-benar habis dicek.

Berdasarkan gambar tangkapan layar yang terlampir, baris masukan pertama diisi dengan nilai 12 534, yang berarti terdapat 12 total data yang akan diketikkan dan angka 534 adalah target pencariannya. Baris masukan selanjutnya memuat deretan 12 angka yang sudah terurut membesar, yaitu 1 3 8 16 32 123 323 323 534 543 823 999. Program mengevaluasi deretan tersebut dengan sangat cepat dan berhasil mendeteksi bahwa angka 534 terletak tepat pada urutan ke-8 (penghitungan posisi di dalam kode pemrograman selalu dihitung mulai dari angka 0, bukan 1), sehingga mencetak keluaran berupa angka 8 sebagai hasil akhir. Seandainya angka target tidak tercantum di dalam deretan masukan tersebut, program sudah disetel untuk mencetak hasil alternatif berupa teks "TIDAK ADA".

D. Kesimpulan

Proses pencarian (searching) merupakan tahapan esensial dalam pemrograman untuk menemukan keberadaan atau posisi suatu informasi spesifik di dalam sekumpulan data. Terdapat dua metode utama yang dapat digunakan, yaitu Sequential Search yang bekerja dengan memeriksa setiap elemen secara berurutan dari awal hingga akhir, serta Binary Search yang beroperasi jauh lebih efisien dengan cara terus membelah dua rentang pencarian. Pemilihan algoritma ini sangat bergantung pada kondisi kumpulan data yang dihadapi, di mana Sequential Search dapat diterapkan pada data acak maupun terurut, sedangkan Binary Search memiliki syarat mutlak berupa kumpulan data yang sudah harus dalam keadaan terurut. Secara keseluruhan, pemahaman dan penerapan metode pencarian yang tepat, baik pada tipe data dasar maupun tipe data bentukan seperti struct, sangat penting untuk mengoptimalkan kinerja program dalam menyeleksi, merekap, dan menemukan informasi secara akurat.